Singapore HDB resale market trends visualization

This is Part I of a two parts Analysis Project.
This notebook studies the Singapore HDB resale market from a **geospatial market-analysis** perspective.
Instead of treating each resale transaction as an isolated row, the workflow consolidates transactions into **building-level signals** that can be mapped across Singapore.

End-to-end workflow

1. Load the official HDB resale transaction dataset.
2. Clean and standardize the raw fields.
3. Merge transactions to a master building list with latitude / longitude.
4. Re-express time into rolling 12-month "analysis years".
5. Aggregate transactions into building-level price and activity metrics.
6. Overlay MRT / LRT lines and compute nearest-station distance.
7. Produce summary tables, static maps, and interactive HTML maps.

Why this notebook is useful

Raw resale tables are difficult to interpret because:

- the same building appears many times across many months,
- citywide spatial patterns are hidden inside tabular data, and
- recent price levels and longer-term growth are easier to compare visually than row by row.

This notebook turns those raw transactions into a set of practical questions:

- Where are HDB resale prices currently highest?
- Which buildings or neighborhoods have seen the strongest five-year appreciation?
- How geographically concentrated are the premium areas?
- How does transport access relate to observed resale patterns?

What to keep in mind while reading

This is an **exploratory and descriptive** notebook. It is designed to help a future reader understand the market, identify patterns worth investigating, and generate hypotheses for later modeling work.
It does **not** try to estimate a causal effect of MRT access, age, or any other feature on price. Instead, it provides a clean visual foundation for that next step.

In [1]:
```
# Optional: install packages the first time you run this notebook
# Uncomment and run if needed.
# %pip install pandas numpy matplotlib geopandas contextily shapely folium branca
```

In [2]:
```
from __future__ import annotations
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from shapely.geometry import LineString, Point
from matplotlib.colors import Normalize, TwoSlopeNorm
import geopandas as gpd
import folium
import contextily as cx
from branca.colormap import LinearColormap

pd.set_option("display.max_columns", 100)
```

In [3]:
```
# ----------------------------
# Configuration
# ----------------------------
```

```
# 1. Data we want to analyze
# HDB_CSV_PATH: local path to the data we want to analyze
# Data Source: https://data.gov.sg/datasets/d_8b84c4ee58e3cfc0ece0d773c8ca6abc/view; click "Download CSV".
# You can also view the column definitions using the link above.
# I have downaded the CSV file for you. The version is "Last updated: 15 Mar 2026, 02:10 SGT". You can use it directl
HDB_CSV = "raw_data/ResaleflatpricesbasedonregistrationdatefromJan2017onwards.csv"

# 2. Prepared data for direct import
HDB_BUILDINGS_MASTER_LIST = "cache/HDB_Buildings_residential.csv"
MRT_GEOCODES = "cache/mrt_geocodes.csv"

# 3. Output files
OUTPUT_BUILDING_METRICS = 'outputs/building_metrics.csv'
OUTPUT_TRANSACTIONS_CLEAN = 'outputs/transactions_clean.csv'
OUTPUT_INTERACTIVE_PSM = "outputs/sg_hdb_psm_interactive.html"
OUTPUT_INTERACTIVE_5Y_CHANGE = "outputs/sg_hdb_5y_change_interactive.html"
```

1) Load HDB resale data

This section ingests the official HDB resale transaction file and converts it from a raw administrative table into an analysis-ready dataset.

What is being prepared here

- parse the transaction month into a true datetime field,
- convert price and floor area columns into numeric variables,
- standardize block and street text,
- create a canonical `address` field for each building, and
- remove incomplete or invalid rows.

Why this section matters

Every later step depends on having a reliable building identifier and clean pricing variables.
If addresses are inconsistent, the same building may be split into multiple groups. If price or area columns remain as strings, medians and comparisons will be incorrect.

Expected outcome

By the end of this section, the table should contain one clean row per resale transaction, with a computed `psm` column that will later be aggregated to the building level.
The output printed below serves as a quick validation that the file schema is correct and that the cleaning step retained a very large usable sample.

In [4]:
```
df_raw = pd.read_csv(HDB_CSV)

# Confirm the downloaded file matches the expected schema from data.gov.sg.
print(df_raw.columns == ["month", "town", "flat_type", "block", "street_name", "storey_range", "floor_area_sqm", "fla
df_raw.head()


def preprocess_hdb_data(df: pd.DataFrame) -> pd.DataFrame:
    """Clean the raw HDB transaction table and create a consistent building key.

    Why this matters:
    - Mapping and aggregation happen at the building level, so addresses must be standardized.
    - Price-per-sqm must be numeric and comparable across rows.
    - Invalid or incomplete rows would otherwise distort later median calculations.
    """
    df = df.copy()

    # Convert dates and numeric columns into analysis-ready types.
    df["month"] = pd.to_datetime(df["month"].astype(str) + "-01", errors="coerce")
    df["floor_area_sqm"] = pd.to_numeric(df["floor_area_sqm"], errors="coerce")
    df["resale_price"] = pd.to_numeric(df["resale_price"], errors="coerce")

    # Normalize text fields so the same building is written in the same way everywhere.
```

```
    df["block"] = df["block"].astype(str).str.strip()
    df["street_name"] = df["street_name"].astype(str).str.strip()

    # Create a building-level join key used later for merging to the master list.
    df["address"] = (
        df["block"] + " " + df["street_name"]
    ).str.replace(r"\s+", " ", regex=True).str.strip()

    # Standardize prices into a comparable intensity metric.
    df["psm"] = round(df["resale_price"] / df["floor_area_sqm"], 0)

    # Remove rows that cannot support later spatial or pricing analysis.
    df = df.dropna(subset=["month", "floor_area_sqm", "resale_price", "psm", "address"])
    df = df[df["floor_area_sqm"] > 0].copy()
    df = df.sort_values(["month", "town", "street_name", "block"]).reset_index(drop=True)
    return df


df = preprocess_hdb_data(df_raw)
print(f"Cleaned rows: {len(df):,}")
```

```
[ True  True  True  True  True  True  True  True  True  True  True]
Cleaned rows: 226,916
```

2) Merge with HDB Residential Buildings Master List (with geocodes)

The resale transactions become much more useful once they are linked to a building master list containing geospatial information.

What is added in this section

Each transaction inherits:

- latitude,
- longitude,
- postal code, and indirectly
- a connection to building-level structural attributes from the master list.

Why the merge is necessary

The original resale file is not directly mappable at the building level.
This merge transforms a transaction table into a spatial dataset and also filters out addresses that no longer appear in the current HDB residential building list.

That filtering is meaningful: some older addresses may have been demolished, converted, or otherwise removed from the current residential stock. Excluding them keeps the map focused on the present-day building inventory.

Why the notebook uses rolling 12-month "years"

The latest file extends into **March 2026**, so calendar-year aggregation would make 2026 a partial year.
Instead, the notebook shifts months into rolling 12-month windows, which means each analysis year represents a full 12-month period and the most recent information is still captured.

Expected outcome

After this step, the data should contain a large set of geocoded transactions tied to current residential buildings, ready for building-level aggregation and mapping.

In [5]:
```
hdb_master_list = pd.read_csv(HDB_BUILDINGS_MASTER_LIST)

# Add a few building attributes that will later be useful for interpretation and mapping.
hdb_master_list['building_age'] = 2026 - hdb_master_list['year_completed']
hdb_master_list['latitude'] = round(hdb_master_list['latitude'], 5)
hdb_master_list['longitude'] = round(hdb_master_list['longitude'], 5)
hdb_master_list['postal'] = hdb_master_list['postal'].astype('object')
hdb_master_list = hdb_master_list[["latitude", "longitude", "postal", "address", "block", "street_name", "max_floor_

# Merge geocodes onto every transaction using the cleaned building address.
```

```
df_geo = df.merge(
    hdb_master_list[["address", "postal", "latitude", "longitude"]],
    on="address",
    how="left",
)

# Keep only buildings that can be confidently located on the map.
df_geo = df_geo.dropna(subset=["postal", "latitude", "longitude"])

# export for future use
df_geo.to_csv(OUTPUT_TRANSACTIONS_CLEAN, index=False)
```

In [6]:
```
# Re-express time into rolling 12-month analysis windows.
df_geo["month"] = pd.to_datetime(df_geo["month"], format="%Y-%m-%d")
analysis_end = df_geo["month"].max()
df_geo["year"] = (df_geo["month"] + pd.DateOffset(months=12 - analysis_end.month)).dt.year

# Drop the earliest partial window so every retained year is a full comparable period.
df_geo = df_geo[df_geo.year > 2017]

print("Geocoded transaction rows:", f"{len(df_geo):,}")
print("Unique buildings:", f"{df_geo['postal'].nunique():,}")
```

```
Geocoded transaction rows: 222,731
Unique buildings: 9,689
```

Function: converting tables into geospatial objects

The helper below converts a normal pandas DataFrame into a GeoDataFrame by creating point geometries from longitude and latitude.

Why this is important

GeoPandas, basemap overlays, nearest-station distance calculations, and spatial plotting all require a geometry column plus a defined coordinate reference system (CRS).
This conversion is the bridge between ordinary tabular analysis and geospatial analysis.

Design choice

The notebook starts in `EPSG:4326` because the master list stores latitude / longitude in geographic coordinates. Later, selected operations are temporarily projected into meters when distance calculations or basemap alignment require it.

In [7]:
```
# Create a GeoDataFrame so buildings can be mapped and used in spatial operations.
def build_geodf(df: pd.DataFrame) -> gpd.GeoDataFrame:
    """Build a point-based GeoDataFrame from longitude/latitude columns."""
    out = df.copy()
    geometry = [Point(xy) for xy in zip(out["longitude"], out["latitude"])]
    gdf = gpd.GeoDataFrame(out, geometry=geometry, crs="EPSG:4326")
    return gdf


gdf_buildings = build_geodf(hdb_master_list)
```

3) Compute building-level statistics

Core idea

Each point on the later maps should represent **one building**, not one individual sale.
This section therefore summarizes the transaction history of each building into a compact set of interpretable indicators.

Metrics created here

- median `psm` by building and analysis year,
- transaction count over the last five analysis years, and
- five-year growth in median price per square meter.

Why this aggregation is useful

The raw dataset is at the transaction level, which is too noisy for a citywide visualization. A building may sell many times, and individual sales can differ because of flat size mix, floor, renovation, or timing.
Using building-level medians produces a more stable signal and answers practical questions such as:

- what is the building's recent market level?
- how active is the building in the resale market?
- how much has pricing changed over time?

Expected outcome

By the end of this section, `gdf_buildings` becomes the notebook's main analytical table: one geocoded row per building with pricing, growth, and activity metrics attached.

In [8]:
```
# Add yearly median PSM for each building. These columns become the base for trend comparisons.
gdf_buildings = gdf_buildings.merge(
    df_geo.groupby(['address', 'year'])['psm'].median().unstack().add_prefix('psm_').reset_index(),
    'left',
    on='address'
)

# Count recent transaction activity as a simple liquidity / market-activity signal.
gdf_buildings = gdf_buildings.merge(
    df_geo[df_geo.year >= 2022].groupby(['address'])['psm'].count().rename('txn_5y'),
    'left',
    on='address'
)

# Compute five-year building-level appreciation from median PSM.
gdf_buildings['psm_growth_5y'] = round(gdf_buildings.psm_2026 / gdf_buildings.psm_2022 - 1, 3) * 100
```

4) Public transport rail (MRT/LRT) overlay

Price maps are much easier to interpret when the public transport network is visible.

What this section adds

- rail station points,
- simplified rail-line geometries built from ordered station coordinates, and
- each building's nearest MRT / LRT station plus straight-line distance.

Why this matters analytically

Distance to rail is one of the most intuitive location variables in Singapore housing analysis. Even without running a formal model, showing the transport network helps a reader visually test whether higher-price areas and stronger-appreciation areas appear to cluster around major rail corridors.

Why distance is calculated in projected coordinates

Latitude / longitude are not measured in meters, so direct distance calculations in geographic coordinates would be misleading. The notebook temporarily reprojects both buildings and stations into Web Mercator (`EPSG:3857`) so nearest-station distances are measured in approximate meters.

Expected outcome

The main building table will now carry both structural market metrics and a transport-accessibility reference, making the later maps much more interpretable.

In [9]:
```
# MRT STATIONS
gdf_rail_station = build_geodf(pd.read_csv(MRT_GEOCODES))

# MRT LINES
mrt_lines = ["NS", "EW", "CC", "CE", "DT", "TE", "NE", "CG", "BP", "SE", "SW", "PE", "PW"]
```

```
rows = []
for colname in mrt_lines:
    tmp = (gdf_rail_station[[colname, "geometry"]].dropna().sort_values(colname))
    rows.append({"RAIL_TYPE": colname, "geometry": LineString(tmp.geometry.tolist())})

gdf_rail_lines = gpd.GeoDataFrame(rows, geometry="geometry", crs=gdf_rail_station.crs)
```

In [10]:
```
# Calculate the nearest MRT/LRT station for each building.
def attach_nearest_station(
    gdf_buildings: gpd.GeoDataFrame,
    gdf_station: gpd.GeoDataFrame,
    station_name_col: str = "NAME",
) -> gpd.GeoDataFrame:
    """Attach nearest-station name and straight-line distance in meters.

    The calculation is done in a projected CRS so Euclidean distance is measured in
    linear units rather than degrees.
    """

    buildings = gdf_buildings.to_crs(epsg=3857).copy()
    stations = gdf_station.to_crs(epsg=3857).copy()

    bxy = np.column_stack([buildings.geometry.x.to_numpy(), buildings.geometry.y.to_numpy()])
    sxy = np.column_stack([stations.geometry.x.to_numpy(), stations.geometry.y.to_numpy()])

    # With ~10k buildings and ~200 stations, a dense distance matrix is still manageable
    # and keeps the logic easy to read.
    dx = bxy[:, None, 0] - sxy[None, :, 0]
    dy = bxy[:, None, 1] - sxy[None, :, 1]
    dist2 = dx * dx + dy * dy

    nearest_idx = dist2.argmin(axis=1)
    nearest_dist_m = np.sqrt(dist2[np.arange(len(buildings)), nearest_idx]).round(0)

    buildings["nearest_station"] = stations.iloc[nearest_idx][station_name_col].to_numpy()
    buildings["stn_distance_m"] = nearest_dist_m

    return buildings.to_crs(epsg=4326)


gdf_buildings = attach_nearest_station(
    gdf_buildings,
    gdf_rail_station,
    station_name_col="NAME",
)
```

In [11]:
```
# Output to CSV files
gdf_buildings[gdf_buildings.txn_5y>0][['postal', 'address', 'block', 'street_name',
       'max_floor_lvl', 'year_completed', 'building_age', 'town',
       'total_dwelling_units', 'psm_2018', 'psm_2019', 'psm_2020',
       'psm_2021', 'psm_2022', 'psm_2023', 'psm_2024', 'psm_2025', 'psm_2026',
       'txn_5y', 'psm_growth_5y', 'nearest_station', 'stn_distance_m']].to_csv(OUTPUT_BUILDING_METRICS, index=False)
```

5) Quick checks and summaries

Before moving into maps, it is useful to look at a few compact summaries.

Why these tables are important

The maps show spatial pattern, but summary tables answer slightly different questions:

- What is the overall market level across Singapore?
- How wide is the spread between lower-priced and upper-priced buildings?
- Which addresses dominate the premium end of the market?
- Which buildings posted the fastest appreciation, and how much of that may reflect a low starting base or small sample size?

Together, these checks anchor the visuals in actual numbers and make it easier for a future reader to interpret the later maps responsibly.

Overall pricing trends across Singapore

This table summarizes the distribution of building-level medians rather than individual transactions.
That is why it is a good reference point for the rest of the notebook: the maps are built from the same building-level units.

What stands out in the current results

- The median building-level price is about **SGD 6,264 per sqm** in the latest analysis window.
- The interquartile range is fairly wide, from roughly **SGD 5,750 to 7,122 per sqm**, showing meaningful location-driven dispersion.
- The median five-year growth is about **30%**, which signals a strong broad-based rise rather than growth concentrated only in the top tail.
- Even so, the top 1% of buildings are far above the rest, especially on the price level measure, which justifies later clipping of map color scales for readability.

In [12]:
```
# overall pricing trends in Singapore
summary = pd.DataFrame({
    "metric": [
        "Price Per Square Meter (PSM): Y2026",
        "% Growth in PSM: Y2022-Y2026",
    ],
    "Bottom 1%": [
        f"${round(gdf_buildings['psm_2026'].quantile(0.01), 0)}",
        f"{round(gdf_buildings['psm_growth_5y'].quantile(0.01), 0)}%",
    ],
    "Bottom 25%": [
        f"${round(gdf_buildings['psm_2026'].quantile(0.25), 0)}",
        f"{round(gdf_buildings['psm_growth_5y'].quantile(0.25), 0)}%",
    ],
    "median": [
        f"${round(gdf_buildings['psm_2026'].median(), 0)}",
        f"{round(gdf_buildings['psm_growth_5y'].median(), 0)}%",
    ],
    "Top 25%": [
        f"${round(gdf_buildings['psm_2026'].quantile(0.75), 0)}",
        f"{round(gdf_buildings['psm_growth_5y'].quantile(0.75), 0)}%",
    ],
    "Top 1%": [
        f"${round(gdf_buildings['psm_2026'].quantile(0.99), 0)}",
        f"{round(gdf_buildings['psm_growth_5y'].quantile(0.99), 0)}%",
    ],
})
summary
```

Out[12]:

	metric	Bottom 1%	Bottom 25%	median	Top 25%	Top 1%
0	Price Per Square Meter (PSM): Y2026	$4731.0	$5750.0	$6264.0	$7122.0	$11785.0
1	% Growth in PSM: Y2022-Y2026	7.0%	23.0%	30.0%	36.0%	55.0%

Top 20 most expensive buildings

This ranking shows where the highest current building-level median PSM values are concentrated.

How to read it

- `psm_2026` is the latest 12-month median price-per-sqm proxy.
- `txn_5y` helps gauge how much recent transaction evidence sits behind that building's estimate.
- `psm_growth_5y` shows how much psm has increased compared to 5 years ago.

Current pattern in the results The most expensive buildings are heavily concentrated in mature and central locations such as **Cantonment Road, Dawson Road, Boon Tiong Road, Henderson Road, Bishan, and parts of Toa Payoh**.
This suggests that the premium tier is geographically clustered rather than evenly spread across Singapore, and many of these buildings also show healthy transaction counts, so the ranking is not driven only by one-off sales.

In [13]:
```
# Top 20 Most Expensive Buildings
gdf_buildings[["postal", "address", "town", "psm_2022", "psm_2026", "psm_growth_5y", "txn_5y"]].dropna().nlargest(20,
```

Out[13]:

	postal	address	town	psm_2022	psm_2026	psm_growth_5y	txn_5y
25	85201	1B CANTONMENT RD	CENTRAL AREA	11452.0	14947.0	30.5	41.0
28	85401	1D CANTONMENT RD	CENTRAL AREA	10280.0	14940.0	45.3	31.0
27	85301	1C CANTONMENT RD	CENTRAL AREA	10794.0	14759.0	36.7	61.0
23	85101	1A CANTONMENT RD	CENTRAL AREA	10274.5	14717.0	43.2	39.0
31	85701	1G CANTONMENT RD	CENTRAL AREA	10389.5	14587.0	40.4	53.0
30	85601	1F CANTONMENT RD	CENTRAL AREA	10936.5	14177.5	29.6	41.0
29	85501	1E CANTONMENT RD	CENTRAL AREA	10633.0	14108.5	32.7	40.0
54	164010	10B BOON TIONG RD	BUKIT MERAH	10787.0	13678.0	26.8	36.0
10097	162009	9A BOON TIONG RD	BUKIT MERAH	10370.5	13643.0	31.6	24.0
10101	142090	90 DAWSON RD	QUEENSTOWN	9793.0	13399.0	36.8	32.0
3532	571273	273A BISHAN ST 24	BISHAN	10323.0	13158.0	27.5	31.0
9881	141086	86 DAWSON RD	QUEENSTOWN	9976.0	13138.5	31.7	90.0
10160	141092	92 DAWSON RD	QUEENSTOWN	9616.0	13094.5	36.2	38.0
51	160010	10A BOON TIONG RD	BUKIT MERAH	10272.0	13043.0	27.0	29.0
1196	312138	138B LOR 1A TOA PAYOH	TOA PAYOH	9415.5	12991.0	38.0	34.0
10124	144091	91 DAWSON RD	QUEENSTOWN	9812.5	12892.0	31.4	31.0
1198	313138	138C LOR 1A TOA PAYOH	TOA PAYOH	9494.5	12585.0	32.6	50.0
10329	151096	96A HENDERSON RD	BUKIT MERAH	10215.0	12557.5	22.9	43.0
1217	312139	139B LOR 1A TOA PAYOH	TOA PAYOH	9214.0	12426.5	34.9	31.0
10098	163009	9B BOON TIONG RD	BUKIT MERAH	10435.0	12424.0	19.1	18.0

Top 20 biggest capital-gain buildings

This ranking focuses on growth rather than current price level.

Why this table matters A building can rank highly on five-year appreciation even if it is not among the most expensive buildings today. That distinction is useful because it separates **current premium** from **recent acceleration**.

Current pattern in the results The strongest five-year gains are much more geographically dispersed than the premium-price list, with examples from **Yishun, Serangoon, Kallang/Whampoa, Bedok, Hougang, Sengkang, Tampines, Jurong West, Punggol, and Woodlands**.
That spread is a valuable signal: recent appreciation has not been limited to the traditional premium central areas.

Caution for interpretation Some of the biggest gainers have relatively low `txn_5y` counts. That does not make the result wrong, but it does mean those buildings deserve a little more caution than highly traded buildings.

In [14]:
```
# Top 20 Biggest Capital Gain Buildings
gdf_buildings[["postal", "address", "town", "psm_2022", "psm_2026", "psm_growth_5y", "txn_5y"]].dropna().nlargest(20,
```

Out[14]:

	postal	address	town	psm_2022	psm_2026	psm_growth_5y	txn_5y
9339	760770	770 YISHUN AVE 3	YISHUN	3429.0	6220.5	81.4	16.0
7012	550524	524 SERANGOON NTH AVE 4	SERANGOON	3587.0	6477.5	80.6	6.0
5114	320039	39 JLN BAHAGIA	KALLANG/WHAMPOA	6038.0	10528.0	74.4	3.0
1333	470146	146 BEDOK RESERVOIR RD	BEDOK	3975.0	6905.0	73.7	8.0
7107	530531	531 HOUGANG AVE 6	HOUGANG	4050.0	7034.0	73.7	6.0
3378	542264	264B COMPASSVALE BOW	SENGKANG	4782.5	8226.0	72.0	10.0
2354	520208	208 TAMPINES ST 21	TAMPINES	4200.0	7105.0	69.2	8.0
1699	530168	168 HOUGANG AVE 1	HOUGANG	4069.0	6800.0	67.1	4.0
4442	400329	329 UBI AVE 1	GEYLANG	4098.0	6786.0	65.6	5.0
1133	470135	135 BEDOK RESERVOIR RD	BEDOK	4466.0	7381.0	65.3	5.0
5793	640434	434 JURONG WEST ST 42	JURONG WEST	3315.0	5481.0	65.3	5.0
10153	640917	917 JURONG WEST ST 91	JURONG WEST	3320.0	5478.0	65.0	20.0
4447	370033	33 BALAM RD	GEYLANG	5106.0	8381.0	64.1	5.0
7681	560605	605 ANG MO KIO AVE 5	ANG MO KIO	4627.0	7584.0	63.9	21.0
8074	410634	634 BEDOK RESERVOIR RD	BEDOK	5069.0	8310.0	63.9	6.0
418	823110	110C PUNGGOL FIELD	PUNGGOL	4273.0	6988.0	63.5	5.0
7850	730617	617 WOODLANDS AVE 4	WOODLANDS	3625.0	5912.0	63.1	11.0
3542	823273	273C PUNGGOL PL	PUNGGOL	5211.0	8483.0	62.8	17.0
1341	350146	146 POTONG PASIR AVE 1	TOA PAYOH	5781.0	9382.5	62.3	12.0
5946	793443	443C FERNVALE RD	SENGKANG	5159.5	8362.0	62.1	23.0

6A) Decision-time views: affordability, capital gain potential, market activity, age, and height

This section generates the main **static PDF-style figures** for reporting.

Why these plots matter

The earlier tables answer "what happened?" in ranking form. The maps answer "where is it happening?"
That spatial view is often much more intuitive, especially for a housing market where location is central to interpretation.

Function rationale: clipping and plotting spatial metrics

The plotting helpers combine several ideas:

- **quantile clipping** so a few outliers do not flatten the rest of the color scale,
- **optional zero-centering** for metrics that can meaningfully be above or below zero,
- **CRS conversion** for proper basemap overlay, and
- **clean PDF-friendly formatting** for presentation use.

Why clip the color scale?

Housing data often contain extreme values. Without clipping, a tiny number of outlier buildings would dominate the legend and make most of Singapore appear visually similar.
Clipping does not change the underlying data; it simply makes the map more readable for the broad middle of the distribution.

In [15]:

```
# Return lower and upper quantile bounds used to clip a metric before mapping.
# This improves contrast for the bulk of the market without changing the source data.
def percentile_clip(series: pd.Series, low: float = 0.1, high: float = 0.9) -> tuple[float, float]:
    """Return quantile clip bounds for a numeric series."""
```

```python
    if isinstance(series, pd.DataFrame):
        series = series.iloc[:, 0]
    s = pd.to_numeric(series, errors="coerce").dropna()
    if len(s) == 0:
        return (np.nan, np.nan)
    return (float(s.quantile(low)), float(s.quantile(high)))


# Create a static choropleth-style point map with optional transport overlays.
def plot_metric_map(
    gdf: gpd.GeoDataFrame,
    metric_col: str,
    title: str,
    cmap: str = "RdYlBu_r",
    clip_quantiles: tuple[float, float] = (0.1, 0.9),
    center_zero: bool = False,
    marker_size: int = 10,
    rail_lines: gpd.GeoDataFrame | None = None,
    station_points: gpd.GeoDataFrame | None = None,
    legend_label: str | None = None,
):
    """Plot a building-level metric on top of a lightweight basemap.

    The metric is clipped to selected quantiles to preserve visual contrast for the
    majority of buildings.
    """

    data = gdf.dropna(subset=[metric_col]).copy()

    q_low, q_high = percentile_clip(data[metric_col], *clip_quantiles)
    data["metric_clipped"] = data[metric_col].clip(q_low, q_high)

    data_3857 = data.to_crs(epsg=3857)
    rail_lines_3857 = rail_lines.to_crs(epsg=3857)
    station_points_3857 = station_points.to_crs(epsg=3857)

    fig, ax = plt.subplots(figsize=(9, 10))

    if center_zero:
        bound = max(abs(q_low), abs(q_high))
        norm = TwoSlopeNorm(vmin=-bound, vcenter=0.0, vmax=bound)
    else:
        norm = Normalize(vmin=q_low, vmax=q_high)

    data_3857.plot(
        ax=ax,
        column="metric_clipped",
        cmap=cmap,
        markersize=marker_size,
        alpha=0.9,
        legend=True,
        norm=norm,
        legend_kwds={"shrink": 0.35, "label": legend_label or metric_col},
        zorder=3,
    )

    rail_lines_3857.plot(ax=ax, color="#444444", linewidth=2.0, alpha=0.45, zorder=1)
    station_points_3857.plot(ax=ax, color="black", markersize=8, alpha=0.35, zorder=2)

    cx.add_basemap(ax, source=cx.providers.CartoDB.Positron, attribution=False)

    ax.set_title(title, fontsize=14)
    ax.set_axis_off()

    # Zoom to the building extent with a small margin so the map fills the page neatly.
    xmin, ymin, xmax, ymax = data_3857.total_bounds
    xpad = (xmax - xmin) * 0.05
    ypad = (ymax - ymin) * 0.05
    ax.set_xlim(xmin - xpad, xmax + xpad)
    ax.set_ylim(ymin - ypad, ymax + ypad)

    plt.tight_layout()
    plt.show()
```

Affordability

This group of visuals focuses on **current price level**, which is the most direct affordability proxy used in the notebook.

What the figures show

1. A national building-level map of current median `psm`.
2. A town-by-year heatmap of median `psm`.
3. A town-by-flat-type heatmap of median resale price for recent years.

How to interpret them together

- The map reveals the **spatial clustering** of expensive and cheaper buildings.
- The town-by-year heatmap shows whether high-price towns have remained persistently expensive or only recently moved up.
- The town-by-flat-type heatmap separates location effects from unit-type effects and shows where larger flats remain especially expensive.

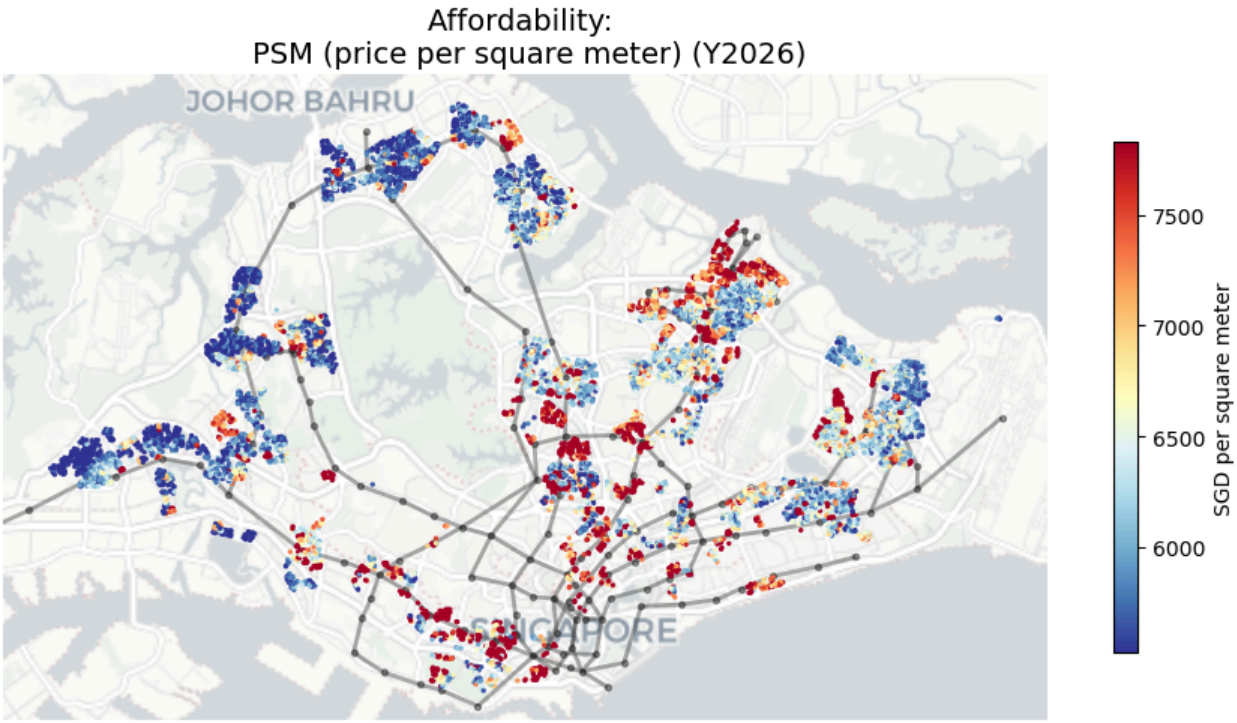
What a future reader should look for

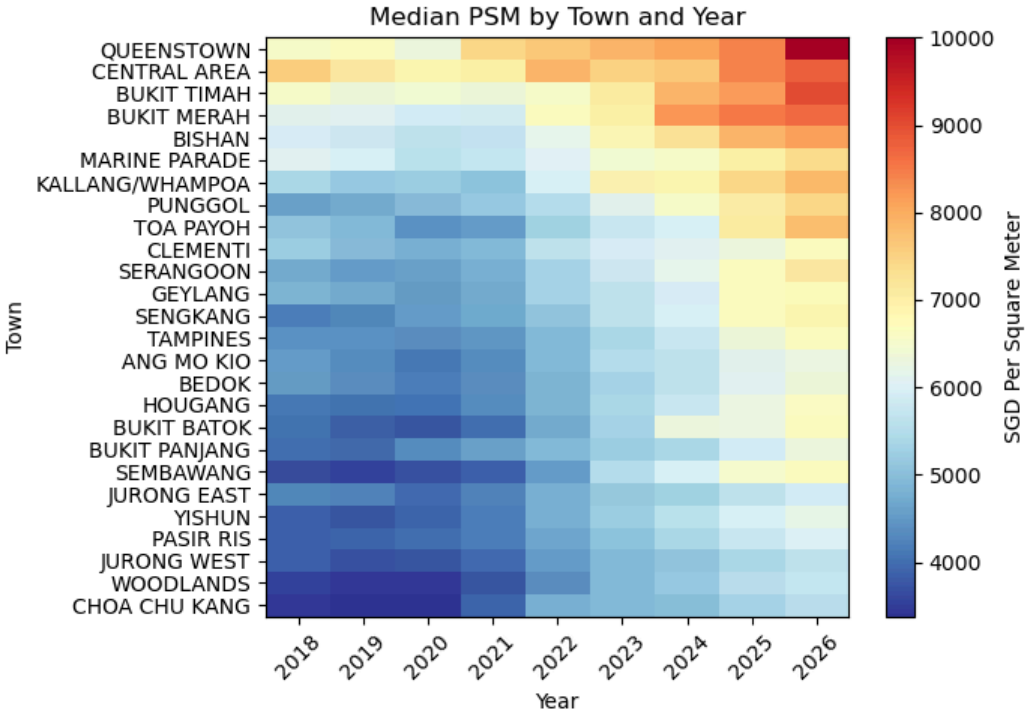
Expect mature central or city-fringe towns to appear as persistent high-price clusters, while outer towns may remain more affordable in level terms even if they later show meaningful growth.

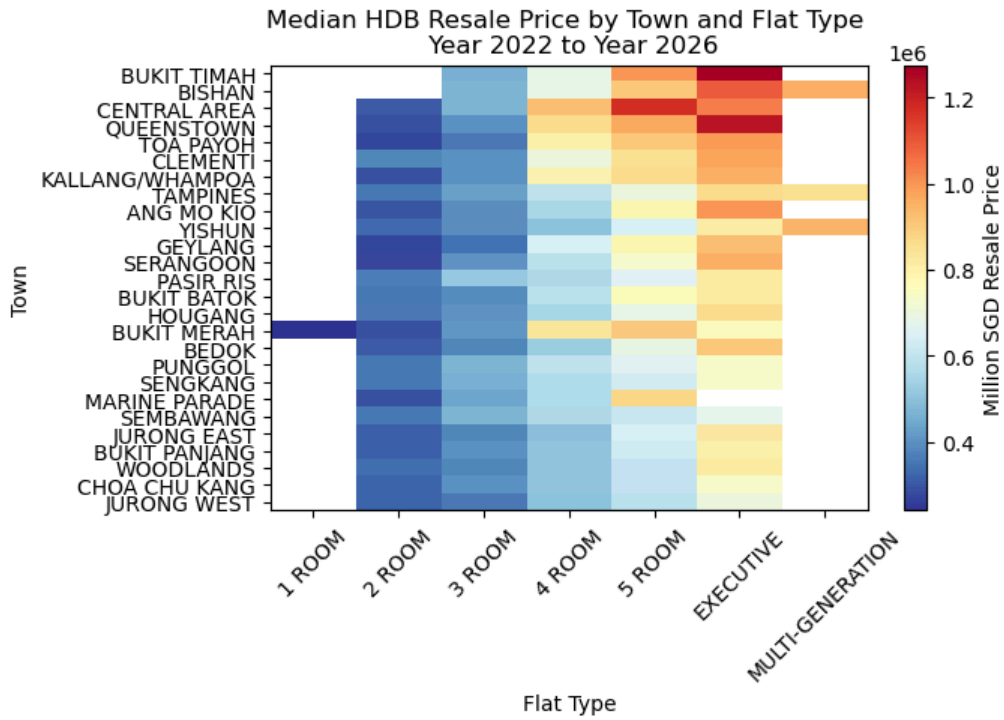
In [16]:

```
plot_metric_map(
    gdf_buildings,
    metric_col="psm_2026",
    title="Affordability: \n PSM (price per square meter) (Y2026)",
    clip_quantiles=(0.15, 0.85),
    center_zero=False,
    marker_size=2,
    rail_lines=gdf_rail_lines,
    station_points=gdf_rail_station,
    legend_label="SGD per square meter",
)

# Median PSM by town and sale year
heatmap = (
    df_geo.groupby(["town", "year"])["psm"]
      .median()
      .unstack()
)

heatmap = heatmap.loc[heatmap.mean(axis=1).sort_values(ascending=False).index]

plt.figure(figsize=(7, 5))
plt.imshow(heatmap, aspect="auto", cmap='RdYlBu_r')
plt.colorbar(label="SGD Per Square Meter")
plt.yticks(range(len(heatmap.index)), heatmap.index)
plt.xticks(range(len(heatmap.columns)), heatmap.columns, rotation=45)
plt.title("Median PSM by Town and Year")
plt.xlabel("Year")
plt.ylabel("Town")
plt.tight_layout()
plt.show()


# Median resale price by town and flat type
heatmap = (
    df_geo[df_geo.year >=2022].groupby(["town", "flat_type"])["resale_price"]
      .median()
      .unstack()
)

heatmap = heatmap.loc[heatmap.mean(axis=1).sort_values(ascending=False).index]

plt.figure(figsize=(7, 5))
plt.imshow(heatmap, aspect="auto", cmap="RdYlBu_r")
plt.colorbar(label="Million SGD Resale Price")
plt.yticks(range(len(heatmap.index)), heatmap.index)
```

```
plt.xticks(range(len(heatmap.columns)), heatmap.columns, rotation=45)
plt.title("Median HDB Resale Price by Town and Flat Type \n Year 2022 to Year 2026")
plt.xlabel("Flat Type")
plt.ylabel("Town")
plt.tight_layout()
plt.show()
```







Capital gain potential

This map shifts attention from price level to **five-year appreciation**.

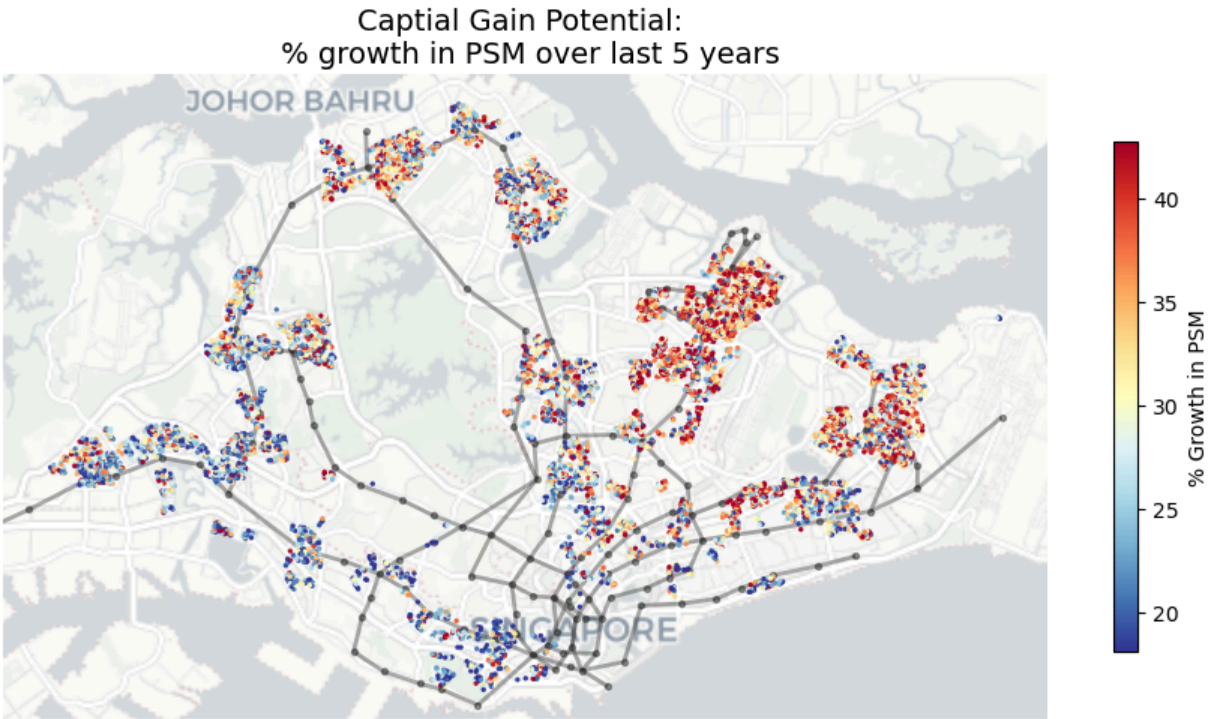
Why this is a separate lens

High current price and high recent growth are not the same thing.
A mature premium area may still be expensive without being the fastest grower, while a more affordable town can post very strong recent gains from a lower base.

What to look for

The current results suggest that fast growth is more geographically dispersed than the premium-price cluster. That is analytically useful because it hints at a broader re-rating of multiple towns rather than a simple strengthening of the already most expensive locations.

In [17]:
```
plot_metric_map(
    gdf_buildings,
    metric_col="psm_growth_5y",
    title="Captial Gain Potential: \n % growth in PSM over last 5 years",
    clip_quantiles=(0.1, 0.9),
    center_zero=False,
    marker_size=2,
    rail_lines=gdf_rail_lines,
    station_points=gdf_rail_station,
    legend_label="% Growth in PSM",
)
```



Most buying and selling

Transaction count is a simple but useful proxy for **market activity and liquidity**.

Why this matters

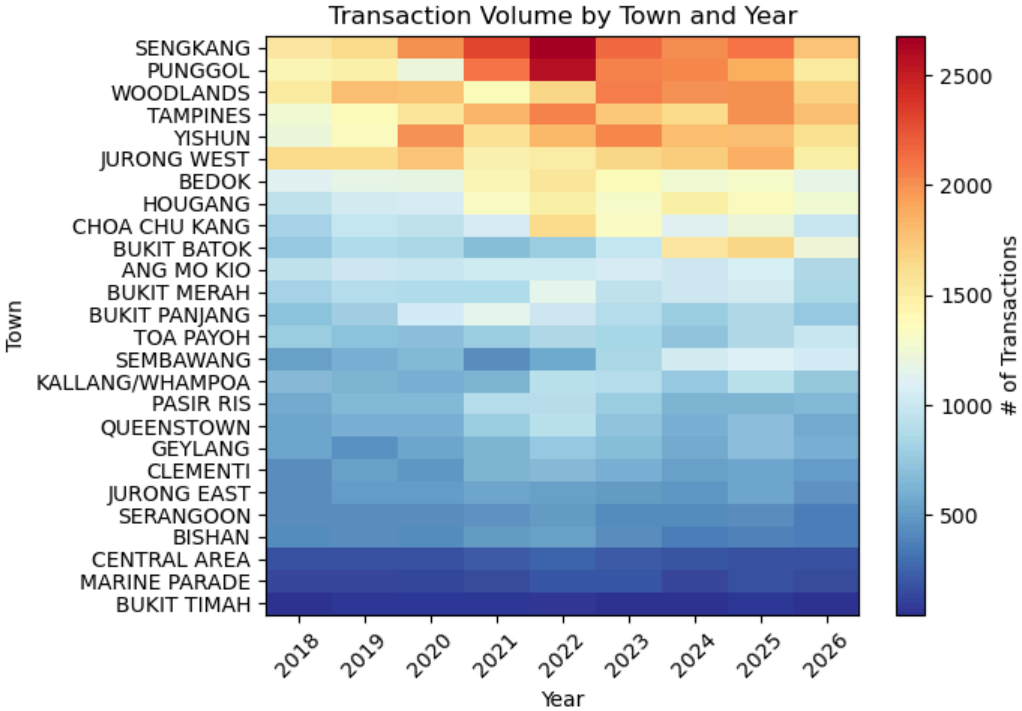
A high-growth or high-price result is easier to trust when it is supported by a healthy number of recent transactions. Conversely, thinly traded buildings can look more volatile because a small number of sales can move the median materially.

How to use this figure

Read this activity map together with the price and growth maps. Areas that are both active and expensive, or active and fast-growing, are especially informative because they are supported by a deeper market signal.

In [18]:
```
plot_metric_map(
    gdf_buildings,
    metric_col="txn_5y",
    title="Most Buying and Selling: \n No. of sales per building over last 5 years",
    clip_quantiles=(0.1, 0.9),
    center_zero=False,
    marker_size=2,
    rail_lines=gdf_rail_lines,
    station_points=gdf_rail_station,
    legend_label="No. of sales per building over last 5 years",
)


# Transaction Volume by town and sale year
heatmap = (
    df_geo.groupby(["town", "year"])["psm"]
      .count()
      .unstack()
)

heatmap = heatmap.loc[heatmap.mean(axis=1).sort_values(ascending=False).index]

plt.figure(figsize=(7, 5))
plt.imshow(heatmap, aspect="auto", cmap='RdYlBu_r')
plt.colorbar(label="# of Transactions")
plt.yticks(range(len(heatmap.index)), heatmap.index)
plt.xticks(range(len(heatmap.columns)), heatmap.columns, rotation=45)
```

```
plt.title("Transaction Volume by Town and Year")
plt.xlabel("Year")
plt.ylabel("Town")
plt.tight_layout()
plt.show()
```



Old buildings vs. new buildings

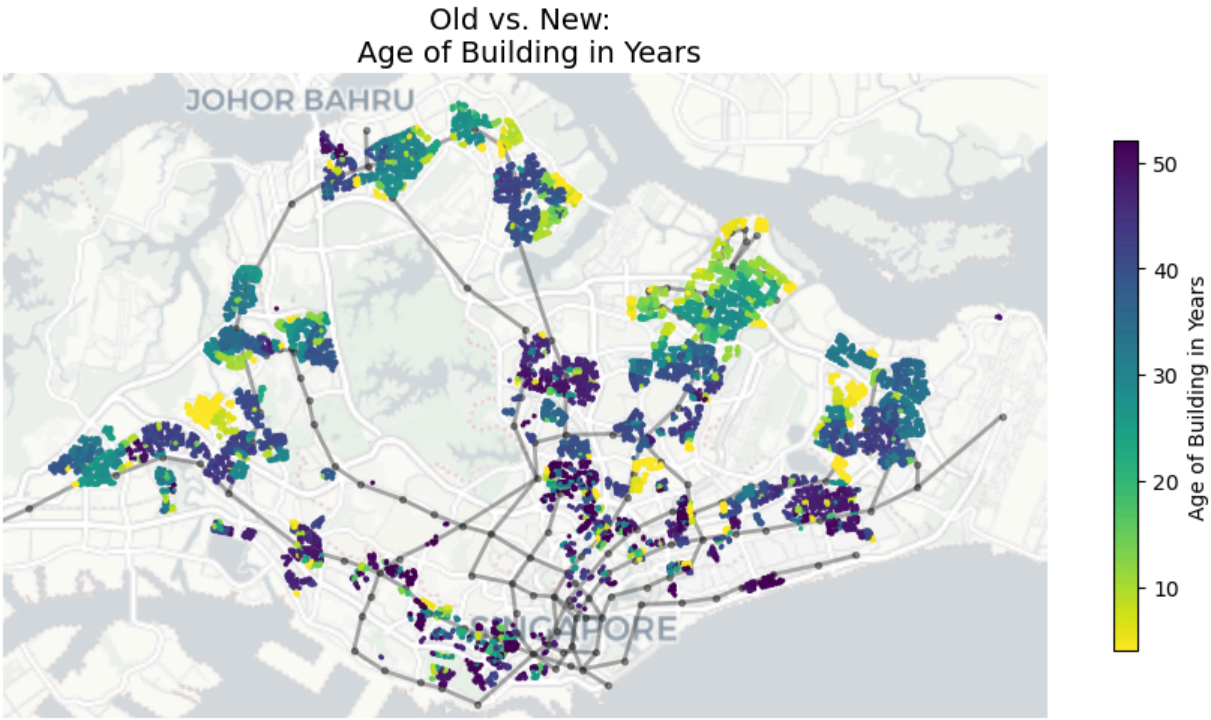
Building age is not a market outcome by itself, but it is one of the most important background variables for interpreting HDB prices.

Why include it here

Older buildings may trade at discounts because of shorter remaining lease, but some older stock in highly central or mature locations can still command strong prices.

Placing age on a map makes it easier to see whether price patterns align more closely with location, age, or a combination of both.

In [19]:
```
plot_metric_map(
    gdf_buildings,
    metric_col="building_age",
    title="Old vs. New: \n Age of Building in Years",
    clip_quantiles=(0.05, 0.95),
    cmap = 'viridis_r',
    center_zero=False,
    marker_size=2,
    rail_lines=gdf_rail_lines,
    station_points=gdf_rail_station,
    legend_label="Age of Building in Years",
)
```



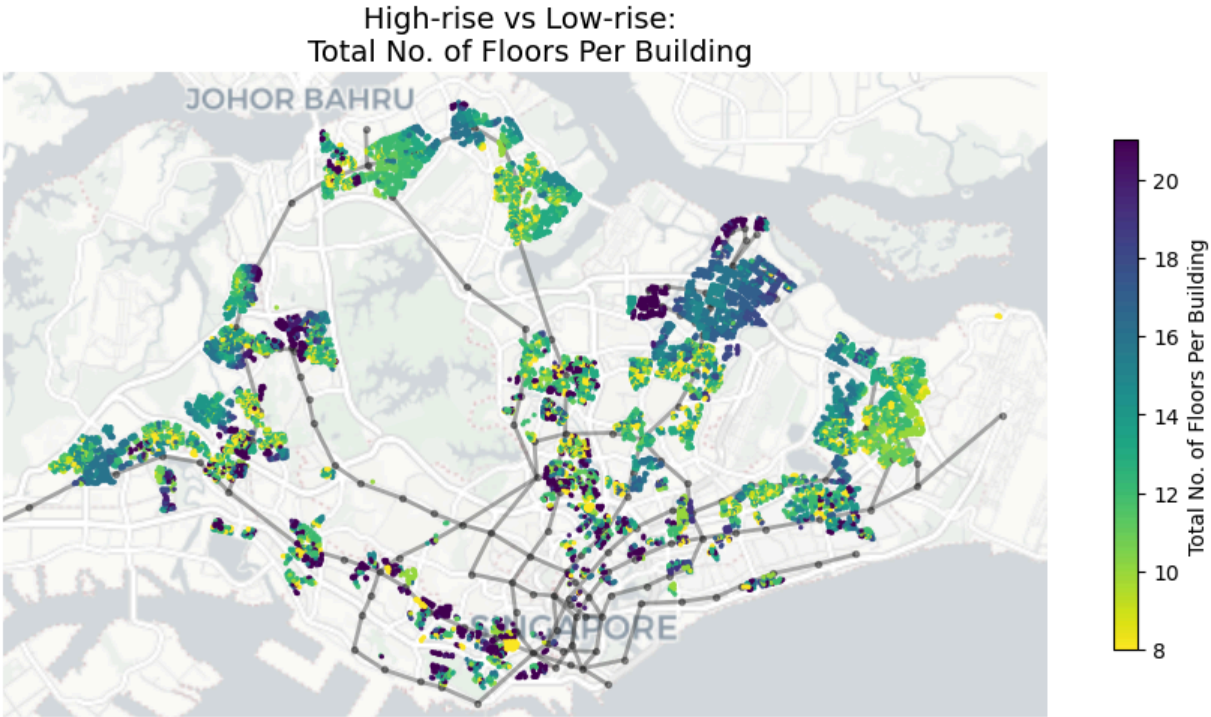
High-rise vs. low-rise

This map provides a simple structural view of Singapore's HDB stock through each building's maximum floor count.

Why it helps interpretation

Height does not determine price on its own, but it helps a reader recognize urban form. High-rise concentrations often correspond to denser and more modern estates, while lower-rise clusters may point to older or differently planned neighborhoods. Seeing this layer next to the price and growth maps gives extra context for why some spatial patterns emerge.

In [20]:
```
plot_metric_map(
    gdf_buildings,
    metric_col="max_floor_lvl",
    title="High-rise vs Low-rise: \n Total No. of Floors Per Building",
    clip_quantiles=(0.1, 0.9),
    cmap = 'viridis_r',
    center_zero=False,
    marker_size=2,
    rail_lines=gdf_rail_lines,
    station_points=gdf_rail_station,
    legend_label="Total No. of Floors Per Building",
)
```



6B) Generate interactive maps (zoomable; hover to view building information)

Static figures are excellent for reporting, but interactive maps are better for exploration and quality checking.

What the interactive maps add

- zoom in and out,
- pan around Singapore,
- inspect individual buildings by hover,
- compare nearby buildings directly, and
- relate local price patterns to MRT / LRT corridors more easily.

Why this section is especially useful

For citywide data, the static map is best for overall pattern recognition, while the interactive map is best for answering local questions such as:

- Which exact building is this point?
- How far is it from the nearest station?
- Is this high-growth point isolated or part of a nearby cluster?

The code below also includes formatting helpers so the tooltip information stays readable for a non-technical user.

In [21]:
```
def fmt_text(value, default: str = "N/A") -> str:
    """Format text values for clean display in HTML tooltips."""
    if pd.isna(value):
        return default
    text = str(value).strip()
    return default if text == "" else text


def fmt_number(value, digits: int = 0, suffix: str = "", default: str = "N/A") -> str:
    """Format numeric values with optional precision and suffix."""
    if pd.isna(value):
        return default
    return f"{value:,.{digits}f}{suffix}"
```

```python
def fmt_int(value, default: str = "N/A") -> str:
    """Format integer-like values for display."""
    if pd.isna(value):
        return default
    return f"{int(round(value)):,}"


def build_tooltip_fields(row: pd.Series) -> list[tuple[str, str]]:
    """Select and format the building attributes shown on hover."""
    pct_text = fmt_number(row.get("psm_growth_5y"), digits=1, suffix="%", default="N/A")
    dist_text = fmt_number(row.get("stn_distance_m"), digits=0, suffix=" m", default="N/A")
    current_psm_text = fmt_number(row.get("psm_2026"), digits=0, suffix=" SGD", default="N/A")
    age_text = fmt_number(row.get("building_age"), digits=0, suffix=" yr", default="N/A")

    fields = [
        ("Address", fmt_text(row.get("address"))),
        ("Postal", fmt_text(row.get("postal"))),
        ("Town", fmt_text(row.get("town"))),
        ("Age", age_text),
        ("PSM (12m)", current_psm_text),
        ("5Y change", pct_text),
        ("Transactions (5y)", fmt_int(row.get("txn_5y"), default="N/A")),
        ("Nearest MRT/LRT", fmt_text(row.get("nearest_station"))),
        ("Distance", dist_text),
    ]
    return fields


def make_tooltip_html(row: pd.Series) -> str:
    """Render tooltip fields into simple HTML."""
    fields = build_tooltip_fields(row)
    return "<br>".join(f"<b>{label}:</b> {value}" for label, value in fields)


def make_interactive_metric_map(
    gdf: gpd.GeoDataFrame,
    metric_col: str,
    map_title: str,
    output_html: Path,
    rail_lines: gpd.GeoDataFrame | None = None,
    station_points: gpd.GeoDataFrame | None = None,
    clip_quantiles: tuple[float, float] = (0.02, 0.98),
    color_scale: list[str] | None = None,
    include_station_markers: bool = True,
    point_radius: float = 4,
):
    """Create an interactive Folium map for a selected building-level metric."""
    data = gdf.dropna(subset=[metric_col]).copy()
    if data.empty:
        raise ValueError(f"No data available for {metric_col}")

    # Keep only columns that are needed for hover text and map rendering so the
    # exported HTML remains lighter and easier to share.
    keep_cols = [
        "address", "postal", "town", "building_age", "psm_2026", "psm_growth_5y", "txn_5y",
        "nearest_station", "stn_distance_m", "latitude", "longitude", "geometry",
        metric_col,
    ]
    keep_cols = [c for c in keep_cols if c in data.columns]
    keep_cols = list(dict.fromkeys(keep_cols))
    data = data[keep_cols].copy()

    q_low, q_high = percentile_clip(data[metric_col], *clip_quantiles)
    data["metric_clipped"] = data[metric_col].clip(q_low, q_high)

    if color_scale is None:
        color_scale = ["#4575b4", "#91bfdb", "#ffffbf", "#fc8d59", "#d73027"]

    sg_center = [data["latitude"].median(), data["longitude"].median()]
    m = folium.Map(location=sg_center, zoom_start=11, tiles="CartoDB positron", control_scale=True)

    title_html = f"""
         <div style="position: fixed;
                     top: 10px; left: 50px; width: 420px; z-index:9999;
```

```
                    background-color: white; padding: 10px 12px; border:2px solid #666;
                    font-size:14px;">
         <b>{map_title}</b>
         </div>
    """
    m.get_root().html.add_child(folium.Element(title_html))

    colormap = LinearColormap(
        colors=color_scale,
        vmin=q_low,
        vmax=q_high,
        caption=map_title,
    )
    colormap.add_to(m)

    lines_for_web = rail_lines
    if lines_for_web is not None and not lines_for_web.empty:
        folium.GeoJson(
            lines_for_web.to_json(),
            name="MRT/LRT lines",
            style_function=lambda feature: {
                "color": "#444444",
                "weight": 2,
                "opacity": 0.55,
            },
            smooth_factor=2,
        ).add_to(m)

    if include_station_markers and station_points is not None and not station_points.empty:
        station_group = folium.FeatureGroup(name="MRT/LRT stations", show=True)
        for _, row in station_points.iterrows():
            folium.CircleMarker(
                location=[row.geometry.y, row.geometry.x],
                radius=2.8,
                color="black",
                weight=1,
                fill=True,
                fill_opacity=0.6,
                tooltip=fmt_text(row.get("NAME")),
            ).add_to(station_group)
        station_group.add_to(m)

    hdb_group = folium.FeatureGroup(name="HDB flats", show=True)

    for _, row in data.iterrows():
        value = row["metric_clipped"]
        color = colormap(value)
        tooltip_html = make_tooltip_html(row)

        folium.CircleMarker(
            location=[row.geometry.y, row.geometry.x],
            radius=point_radius,
            stroke=False,
            fill=True,
            fill_color=color,
            fill_opacity=0.75,
            tooltip=folium.Tooltip(tooltip_html, sticky=True),
        ).add_to(hdb_group)

    hdb_group.add_to(m)

    folium.LayerControl(collapsed=False).add_to(m)

    m.save(str(output_html))
    return m
```

In [22]:

```
interactive_psm_map = make_interactive_metric_map(
    gdf_buildings,
    metric_col="psm_2026",
    map_title="Singapore HDB resale market — median price per sqm by building (latest 12 months) - Interactive",
    output_html=OUTPUT_INTERACTIVE_PSM,
    rail_lines=gdf_rail_lines,
    station_points=gdf_rail_station,
```

```
    clip_quantiles=(0.15, 0.85)
)
```

In [23]:
```
interactive_change_map = make_interactive_metric_map(
    gdf_buildings,
    metric_col="psm_growth_5y",
    map_title="Singapore HDB resale market — 5-year % change in median price per sqm by building - Interactive",
    output_html=OUTPUT_INTERACTIVE_5Y_CHANGE,
    rail_lines=gdf_rail_lines,
    station_points=gdf_rail_station,
    clip_quantiles=(0.15, 0.85),
)
```

In [24]:
```
print("Interactive HTML files written to:")
print(" -", OUTPUT_INTERACTIVE_PSM)
print(" -", OUTPUT_INTERACTIVE_5Y_CHANGE)
```

```
Interactive HTML files written to:
 - outputs/sg_hdb_psm_interactive.html
 - outputs/sg_hdb_5y_change_interactive.html
```

7) Notes for interpretation

A few cautions are important when reading the results.

What these metrics are — and are not

- These are **transaction-based medians**, not appraised values and not causal estimates.
- Buildings with low transaction counts are inherently noisier than heavily traded buildings.
- Apparent price differences reflect a mix of location, flat type mix, floor effects, remaining lease, timing, and other unmodeled factors.
- The five-year comparison is descriptive and should be interpreted as a market trend indicator, not a controlled estimate of appreciation drivers.

How to use this notebook well

The best use of this notebook is to:

1. identify spatial patterns,
2. compare relative positioning of towns and buildings,
3. generate hypotheses about accessibility, age, and market activity, and
4. prepare features or questions for a later predictive model.

That is why this notebook works best as the **visual and exploratory companion** to a separate modeling notebook. It tells you where to look and what patterns may matter; the modeling stage can then test those patterns more systematically.